



Pentesting Contracts

Introduction

In the world of offensive security, the difference between a cybercriminal and a professional penetration tester is often a single document: **The Contract**. Before any scanning tool is run or any packet is sent, a legal and technical framework must be established. This framework defines the **Rules of Engagement**, protects both the client and the tester, and ensures that the exercise provides real value rather than just chaos. The purpose of this concept page is to guide you through the critical components of Pentesting Contracts, from the initial legal agreements to the technical scoping that defines your boundaries.

The Get Out of Jail Free Card

Authorization is the most critical concept in penetration testing. Without explicit, written permission to test a specific target, you are committing a crime. A Pentesting Contract is not just about getting paid; it is your **legal shield**. It proves that your actions, which might otherwise be interpreted as malicious attacks, are authorized, requested, and conducted within agreed-upon boundaries.

Core Contractual Documents

Professional engagements typically involve several distinct documents, each serving a specific purpose:

1. Non-Disclosure Agreement (NDA)

The Silence Agreement

Before discussing sensitive details about a company's infrastructure, an NDA is signed.

- **Purpose:** Protects confidential information shared during negotiations and testing.
- **Key Focus:** Ensuring that vulnerabilities found (and client data accessed) are never leaked.

2. Master Services Agreement (MSA)

The Relationship Agreement

This is the high-level contract that governs the ongoing business relationship between the testing firm and the client.

- **Purpose:** Establishes general terms like payment cycles, liability caps, dispute resolution, and intellectual property rights.
- **Key Focus:** Legal and financial protection. It often covers multiple future engagements.

3. Statement of Work (SoW)

The Job Description

The SoW is specific to a single project. If an MSA covers the **relationship**, the SoW covers the **task**.

- **Purpose:** Defines exactly what will be done, for how much, and by when.
- **Key Focus:**



- **Deliverables:** What will the client get? A PDF report, a re-test.
 - (/) ◦ **Timeline:** Start date, end date, and milestones.
 - **Price:** Fixed cost or hourly rate.
-

4. Rules of Engagement (RoE)

The Battle Plan

This is the most technical and critical document for the tester. It translates legal boundaries into technical constraints.

- **Purpose:** Defines How the test will be conducted.
- **Key Focus:**
 - **Allowed/Disallowed Attack Vectors:** Can we use Social Engineering? Can we attempt DoS (Denial of Service)?
 - **Testing Windows:** Are we only testing between 9 PM and 5 AM?
 - **Emergency Contact:** Who do we call if we accidentally bring down the production server?

Scoping

What is Scoping?

Scoping is the process of identifying exactly **what** is to be tested. It is the most common source of disputes. A poor scope can lead to a tester attacking the wrong IP address (illegal) or missing the critical vulnerability because it was **out of bounds**.

Scoping Categories

Blackbox (Zero Knowledge) :

The tester is given no information other than the company name or a URL.

- **Simulation:** Real-world external attacker.
- **Pros:** Realistic.
- **Cons:** Time-consuming; might miss internal issues.

Whitebox (Full Knowledge) :

The tester is given full access: source code, network diagrams, and credentials.

- **Simulation:** Insider threat or thorough audit.
- **Pros:** Comprehensive; finds the most bugs.
- **Cons:** Less **realistic** simulation of an outsider.

Graybox (Partial Knowledge) :

A mix. The tester might get user credentials but no source code.

- **Simulation:** Attacker who has breached a user account.
- **Pros:** Balanced approach; most common in modern pentesting.

Common Scoping Pitfalls (Scope Creep)

Scope Creep

happens when the testing expands beyond the original agreement without adjustments to time or budget.

- **Example:** While you're testing the web app, can you also quick-check our email server?
- **Solution:** Always require a formal Change Order or revised SoW for new targets.

Third-Party Constraints



Cloud Providers (AWS, Azure, GCP) :

(/)

You generally cannot test the cloud provider's infrastructure (trying to exploit an AWS router). You test *your* assets hosted on them. Historic requirements for permission forms have relaxed, but policy violations (like DDoS) remain strictly banned.

SaaS Providers :

If a client uses a third-party CRM (like Salesforce), you usually **cannot** test it without the SaaS provider's explicit permission, which is rarely granted.

Liability and Indemnification

Who is responsible if things break?

Penetration tests are inherently risky. Systems crash. Data gets corrupted.

Indemnification Clauses :

A specialized clause in the MSA usually states that the Client will indemnify and hold harmless the Tester for damages caused by *authorized* testing activities, provided the Tester followed the Rules of Engagement.

- **Translation:** If I run the Nmap scan you asked for, and your old server crashes because it couldn't handle the traffic, you can't sue me.

Gross Negligence :

However, contracts rarely protect against **negligence**.

- **Example:** If the RoE says No DoS attacks and the tester runs a stress test that crashes the network, the tester is liable.

Conclusion

Pentesting contracts are the foundation of professional security work. They transform what would be a felony into a valuable business service. By establishing clear authorization through an MSA, SoW, and RoE, you protect yourself legally and ensure the client gets the testing they need without unexpected damages. The key takeaway is simple: **Technical skills get you the job, but contracts keep you out of jail**. Mastering these agreements is not just administrative work; it is the first and most important step of every engagement.

Good luck, Pentester! Remember: No contract, no test. No code, no safety net.

